q4-colab

April 3, 2025

1 Assignment 2 - Question 4: Convolutional Neural Network - Colab Version

Course Name: Machine Learning (DDA3020)

Please enter your personal information (Double-click this block first)

Name: Chen Ziqi

Student ID: 123090059

It's highly recommended to finish Question 3 first.

1.0.1 Overview

In this question, you will implement two CNN models and train them on the same dataset as Question 3 (Fasion-MNIST). We will discover how well-suited CNNs are for intensive data tasks such as image processing, compared to traditional machine learning algorithms (like those tree tree-based models in Question 3). Similarly, your task is to **run all codes in this script and complete the parts marked with** [TASK].

1.0.2 Introduction of TensorFlow

TensorFlow is a powerful open-source package for machine learning and deep learning, enabling efficient implementation of complex models like neural networks with ease. By using this version on Colab, you don't need to install any additional packages.

To check whether the package is successfully installed, you can try to run the following import block.

[ ]:
```
import numpy as np
import gzip
import numpy as np
import random
import os
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.utils import to_categorical
```

Don't need to carefully read this block since it's just loading the dataset. Just run it.

[ ]:
```
def load_mnist(path, kind, subset=None):
    labels_path = os.path.join(path, '%s-labels-idx1-ubyte.gz'%kind)
    images_path = os.path.join(path, '%s-images-idx3-ubyte.gz'%kind)

    with gzip.open(labels_path, 'rb') as lbpath:
        labels = np.frombuffer(lbpath.read(), dtype=np.uint8, offset=8)
    with gzip.open(images_path, 'rb') as imgpath:
        images = np.frombuffer(imgpath.read(), dtype=np.uint8, offset=16).
 ↪reshape(len(labels), 784)

    if subset is not None:
        selected_images, selected_labels = [], []
        for label in range(10):
            indices = np.where(labels == label)[0]
            selected_indices = np.random.choice(indices, subset, replace=False)
            selected_images.append(images[selected_indices])
            selected_labels.append(labels[selected_indices])
        images = np.concatenate(selected_images, axis=0)
        labels = np.concatenate(selected_labels, axis=0)

        paired = list(zip(images, labels))
        random.shuffle(paired)
        images, labels = zip(*paired)

    return np.array(images), np.array(labels)
```

In this question, we will use all data of Fashion-MNIST and do a little bit data preprocessing. Here we add some addtional code to help download the data and put it on the right position, so that we don't need to manually upload it every time.

[ ]:
```
! mkdir data
! wget -q -P ./data/ http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/
 ↪train-images-idx3-ubyte.gz
! wget -q -P ./data/ http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/
 ↪train-labels-idx1-ubyte.gz
! wget -q -P ./data/ http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/
 ↪t10k-images-idx3-ubyte.gz
! wget -q -P ./data/ http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/
 ↪t10k-labels-idx1-ubyte.gz
```

```
mkdir: cannot create directory ‘data’: File exists
```

[ ]:
```
X_train, y_train = load_mnist('./data/', kind='train')
X_test, y_test = load_mnist('./data/', kind='t10k')

X_train = X_train.reshape((X_train.shape[0], 28, 28, 1)).astype('float32') / 255
X_test = X_test.reshape((X_test.shape[0], 28, 28, 1)).astype('float32') / 255
```

```
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

If you are familiar with TensorFlow or PyTorch, you can straightly start Task 1. Otherwise, it's highly recommended to start from https://www.tensorflow.org/tutorials/images/cnn to gain some basic capacity on building neural network by TensorFlow.

1.0.3 Task 1

At the beginning, we need to build a very simple CNN model with the structure of 1. A 2D convolutional layer with 16 filters with each size 3*3 (RELU activation function) 2. A 2D maxpooling layer with 2*2 pooling window 3. A flatten layer to convert 2D feature into 1D vector 4. A fully connected layer using Softmax activation

Remember that we are doing a image classification task, so we shall use categorical cross entropy function as the loss funtion. [TASK] (10 points)

[ ]:
```
##########################
## Write your code here ##
##########################

model = models.Sequential([
    layers.Conv2D(16, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train, y_train, epochs=2, batch_size=32, validation_split=0.1)  #␣
 ↪Train the model
test_loss, test_acc = model.evaluate(X_test, y_test)                        # ␣
 ↪Test the model
```

```
/usr/local/lib/python3.11/dist-
packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Model: "sequential"
```

```
Layer (type)                         Output Shape                ␣
↪Param #

conv2d (Conv2D)                      (None, 26, 26, 16)          ␣
↪160

max_pooling2d (MaxPooling2D)         (None, 13, 13, 16)          ␣
↪  0

flatten (Flatten)                    (None, 2704)                ␣
↪  0

dense (Dense)                        (None, 10)                  ␣
↪27,050


Total params: 27,210 (106.29 KB)


Trainable params: 27,210 (106.29 KB)


Non-trainable params: 0 (0.00 B)


Epoch 1/2
1688/1688             22s 12ms/step -
accuracy: 0.7819 - loss: 0.6518 - val_accuracy: 0.8737 - val_loss: 0.3563
Epoch 2/2
1688/1688             42s 13ms/step -
accuracy: 0.8798 - loss: 0.3423 - val_accuracy: 0.8888 - val_loss: 0.3169
313/313             2s 8ms/step -
accuracy: 0.8879 - loss: 0.3261
```

1.0.4 Task 2

Then we will take a challenge to implement a more complex CNN model to have a better classification performance. Here is a structure for your reference, or you can also design your own CNN model. The only requirement is to have a better performance than the simple CNN in Task 1 (a larger accuracy score on test set).

The reference structure is devided into three parts: 1. Primary Feature Extraction Part 1. A 2D convolutional layer with 32 filters with each size 3*3 (RELU activation function) 2. A normalization layer 3. A 2D maxpooling layer with 2*2 pooling window 4. A dropout layer (randomly drops 25% of units) (designed for preventing overfitting) 2. Advanced Feature Extraction Part

```
This part is mostly similar to the previous section. The only difference is to use more filters
```

3. Classification Part

1. A flatten layer to convert 2D feature into 1D vector
2. A fully connected layer with 512 units using RELU to summerize high-dimensinal features
3. Another connected layer for Softmax classification

Remember that we are doing a image classification task, so we shall use categorical cross entropy function as the loss funtion. [TASK] (10 points)

[ ]:
```
##########################
## Write your code here ##
##########################

model = models.Sequential()

model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.BatchNormalization())
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Dropout(0.25))

model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.BatchNormalization())
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Dropout(0.25))

model.add(layers.Flatten())
model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dropout(0.5))
model.add(layers.Dense(10, activation='softmax'))

model.compile(optimizer='adam',
                    loss='categorical_crossentropy',
                    metrics=['accuracy'])

model.fit(X_train, y_train, epochs=2, batch_size=32, validation_split=0.1)
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"Test accuracy: {test_acc:.4f}")
```

```
Epoch 1/2
1688/1688             102s 59ms/step
- accuracy: 0.7552 - loss: 0.7894 - val_accuracy: 0.8600 - val_loss: 0.3757
Epoch 2/2
1688/1688             140s 58ms/step
- accuracy: 0.8571 - loss: 0.3954 - val_accuracy: 0.8673 - val_loss: 0.3515
313/313             5s 16ms/step -
accuracy: 0.8641 - loss: 0.3726
Test accuracy: 0.8612
```